

08

강

R 컴퓨팅

R 프로그래밍의 핵심구조 (1)

통계·데이터과학과 장영재 교수

학습목차

1 프로그래밍 언어로서의 R

2 연산자

3 기본적인 R 함수

4 실습하기

학습목표

- 1 프로그래밍 언어로서의 R의 특징을 이해할 수 있다.
- 2 R 연산자의 특징을 이해하고 활용할 수 있다.
- 3 R 내장 함수의 특징을 이해하고 활용할 수 있다.



1. 프로그래밍 언어로서의 R

- ▶ R은 반복문, 조건문 등을 이용하여 다양한 프로그래밍이 가능한 프로그래밍 언어
 - 기본적인 프로그래밍 구조를 적절히 활용하여 효과적인 작업수행을 위한 도구를 만들어 내는 것이 R 사용자의 목표

02

연산자

2. 연산자

▶ 산술연산자

예제 5-1 수치형 자료의 사칙연산

① 덧셈연산자 : +

> 2+3 # 일반적인 숫자의 덧셈

[1] 5

> x <- 2 # 값을 변수에 할당한 후에 덧셈

> y <- 3

> x+y

[1] 5

> v1 <- c(1,3) # 벡터 v1을 정의

> v2 <- c(5,7) # 벡터 v2를 정의

> v1+v2 # 벡터의 덧셈

[1] 6 10

2. 연산자

▶ 산술연산자

예제 5-1 수치형 자료의 사칙연산

```
> A <- matrix(c(1,2,3,4), ncol=2)      # 행렬 A를 정의
> B <- matrix(c(3,4,5,6), ncol=2)      # 행렬 B를 정의
> A+B                                    # 행렬의 덧셈
```

```
[,1] [,2]
```

```
[1,] 4 8
```

```
[2,] 6 10
```

② 뺄셈연산자 : -

```
> 5-3
```

```
[1] 2
```

```
> x <- 5
```

```
> y <- 3
```


2. 연산자

▶ 산술연산자

예제 5-1 수치형 자료의 사칙연산

```
> x-y
```

```
[1] 2
```

```
> v1 <- c(5,7)
```

```
# 벡터 v1을 정의
```

```
> v2 <- c(1,3)
```

```
# 벡터 v2를 정의
```

```
> v1-v2
```

```
# 벡터의 뺄셈
```

```
[1] 4 4
```

```
> A <- matrix(c(1,2,3,4), ncol=2)
```

```
# 행렬 A를 정의
```

```
> B <- matrix(c(3,4,5,6), ncol=2)
```

```
# 행렬 B를 정의
```

```
> B-A
```

```
# 행렬의 뺄셈
```

```
[,1] [,2]
```

```
[1,] 2 2
```

```
[2,] 2 2
```

2. 연산자

③ 곱셈연산자 : *

```
> 3*5
```

```
[1] 15
```

```
> x <- 5
```

```
> y <- 3
```

```
> x*y
```

```
[1] 15
```

```
> v1 <- c(1,3)           # 벡터 v1을 정의
```

```
> v2 <- c(5,7)           # 벡터 v2를 정의
```

```
> v1*v2                   # 곱셈연산자 *에 의한 곱셈은 각 벡터의 동일 위치 원소 간의 곱
```

```
[1] 5 21
```

2. 연산자

③ 곱셈연산자 : *

> A <- matrix(c(1,2,3,4), ncol=2) # 행렬 A를 정의

> B <- matrix(c(3,4,5,6), ncol=2) # 행렬 B를 정의

> A*B # 곱셈연산자 *에 의한 곱셈은 각 행렬의 동일 위치 원소 간의 곱

[,1] [,2]

[1,] 3 15

[2,] 8 24

2. 연산자

④ 나눗셈연산자 : /

```
> 5/2
```

```
[1] 2.5
```

```
> x <- 5
```

```
> y <- 2
```

```
> x/y
```

```
[1] 2.5
```

```
> v1 <- c(4,6)
```

```
# 벡터 v1을 정의
```

```
> v2 <- c(2,3)
```

```
# 벡터 v2를 정의
```

```
> v1/v2
```

```
# 각 벡터의 동일 위치 원소 간의 나눗셈
```

```
[1] 2 2
```

2. 연산자

④ 나눗셈연산자 : /

```
> A <- matrix(c(1,2,3,4), ncol=2)
```

```
# 행렬 A를 정의
```

```
> B <- matrix(c(3,4,5,6), ncol=2)
```

```
# 행렬 B를 정의
```

```
> B/A
```

```
# 각 행렬의 동일 위치 원소 간의 나눗셈
```

```
[,1] [,2]
```

```
[1,] 3 1.666667
```

```
[2,] 2 1.500000
```

2. 연산자

예제 5-2 수치형 원소로 이루어진 행렬의 제곱, 정수 나눗셈 및 행렬의 곱셈과 관련된 예제

① 제곱연산자 : ^

```
> 2^4
```

일반적인 수치형 자료의 제곱 2의 네 제곱

```
[1] 16
```

```
> A <- matrix(c(1,2,3,4), ncol=2)
```

행렬 A를 정의

```
> B <- matrix(c(2,2,2,2), ncol=2)
```

행렬 B를 정의

```
> A^B
```

행렬의 각 원소 간 제곱연산 수행

```
[,1] [,2]
```

```
[1,] 1 9
```

```
[2,] 4 16
```

2. 연산자

② 정수 나눗셈연산자 : %/%

```
> x <- 3
```

```
> y <- 2
```

```
> x%/%y
```

```
[1] 1
```

```
> A <- matrix(c(1,2,3,4), ncol=2)
```

```
> B <- matrix(c(3,4,5,6), ncol=2)
```

```
> B%/%A
```

```
[,1] [,2]
```

```
[1,] 3 1
```

```
[2,] 2 1
```

정수 나눗셈은 몫의 정수 부분만을 출력

나눗셈 결과는 1.5지만, 정수값인 1만을 출력

행렬 A를 정의

행렬 B를 정의

행렬의 각 원소 간 정수 나눗셈을 실시

2. 연산자

③ 행렬의 곱셈연산자 : `%*%`

```
> A <- matrix(c(5,10,2,1), ncol=2)
```

```
> B <- matrix(c(3,4,5,6), ncol=2)
```

```
> A%*%B
```

```
[1] [2]
```

```
[1,] 23 37
```

```
[2,] 34 56
```

▶ 비교연산자와 논리연산자

- 비교연산자는 대상이 되는 두 객체를 비교하여 비교의 결과가

참인지 거짓인지를 밝히는 연산을 수행하며 결과값으로 논리값을 출력

- 논리연산자는 논리값을 결합하여 논리 구조를 설정하는 연산을 수행

2. 연산자

예제 5-3 비교연산자와 논리연산자의 사용법

① 비교연산자

`==` : 비교되는 두 항이 같은지를 비교. 같을 경우 True, 다를 경우 False

`!=` : 비교되는 두 항이 다른지를 비교. 같을 경우 False, 다를 경우 True

`<=` : 왼쪽 항이 오른쪽 항보다 작거나 같은지를 판단. 작거나 같으면 True, 크면 False

`<` : 왼쪽 항이 오른쪽 항보다 작은지 판단. 작으면 True, 크면 False

`>` : 왼쪽 항이 오른쪽 항보다 큰지 판단. 크면 True, 작으면 False

`>=` : 왼쪽 항이 오른쪽 항보다 크거나 같은지 판단. 크거나 같으면 True, 작으면 False

```
> x <- 2
> 2 == 2
[1] TRUE
> y <- 3
> x == y
[1] FALSE
```

2. 연산자

예제 5-3 비교연산자와 논리연산자의 사용법

```
> x <- 3
> 1 != 2
[1] TRUE
```

```
> y <- 3
> x != y
[1] FALSE
```

```
> x <- 2
> 1 <= 2
[1] TRUE
```

```
> x <- 3
> y <- 2
> x <= y
[1] FALSE
```

```
> x <- 3
> 1 < 2
[1] TRUE
```

```
> y <- 2
> x < y
[1] FALSE
```

2. 연산자

예제 5-3 비교연산자와 논리연산자의 사용법

	> x <- 3	
> 1 > 2	> y <- 2	
[1] FALSE	> x > y	
	[1] TRUE	
	> x <- 2	> x <- 3
> 1 >= 2	> y <- 2	> y <- 2
[1] FALSE	> x >= y	> x >= y
	[1] TRUE	[1] TRUE

2. 연산자

② 논리연산자

&&, & : &&는 스칼라 and 논리연산자이고, &는 벡터에서의 and 논리연산자
 ||, | : ||는 스칼라 or 논리연산자이고, |는 벡터에서의 or 논리연산자

```
> 1 == 1 && 2 > 3
```

```
[1] FALSE
```

```
> 2 == 2 && 4 > 3
```

```
[1] TRUE
```

```
> 1 == 1 && c(3 == 3, 2 > 3)
```

```
[1] TRUE
```

경고메시지(들):

1 == 1 && c(3 == 3, 2 > 3)에서:

'length(x) = 2 > 1' in coercion to 'logical(1)'

```
> 1 == 1 & c(3 == 3, 2 > 3)
```

```
[1] TRUE FALSE
```

2. 연산자

② 논리연산자

```
> 1 == 1 || 3 > 2
```

```
[1] TRUE
```

```
> 2 != 2 || 3 > 2
```

```
[1] TRUE
```

```
> 2 != 2 || 3 > 4
```

```
[1] FALSE
```

```
> 2 != 2 || c(2 == 2, 3 > 4)
```

```
[1] TRUE
```

경고메시지(들):

2 != 2 || c(2 == 2, 3 > 4)에서:

'length(x) = 2 > 1' in coercion to 'logical(1)'

```
> 2 != 2 | c(2 == 2, 3 > 4)
```

```
[1] TRUE FALSE
```

2. 연산자

▶ 집합연산자

- 벡터를 원소들로 이루어진 집합으로 볼 때, 수학적인 정의에 따른 다양한 집합연산을 실시할 수 있음

예제 5-4 주요 집합연산자의 사용법

- union과 intersect : `union(x,y)`는 집합 x와 집합 y의 합집합을 구하고,
`intersect(x,y)`는 집합 x와 집합 y의 교집합을 산출
- setdiff와 setequal : `setdiff(x,y)`는 집합 x와 집합 y의 차집합을 구하고,
`setequal(x,y)`는 집합 x와 집합 y가 같은지 여부를 판단
- %in%와 choose(n,k) : `c%in%x`는 원소 c가 집합 x에 속하는지를 판단하고 `choose(n,k)`는
n개의 원소로 이루어진 집합에서 추출가능한 k개의 원소로 이루어진 부분집합의 수를 계산

2. 연산자

예제 5-4 주요 집합연산자의 사용법

```
> x <- c(1,2,5)
> y <- c(5,1,7,8)
> intersect(x,y)
[1] 1 5
> union(x,y)
[1] 1 2 5 7 8
```

집합 x와 집합 y는 위의 예에서 정의된 집합과 동일하다고 가정하면,

```
> setdiff(x,y)
[1] 2
> setdiff(y,x)
[1] 7 8
> setequal(x,y)
[1] FALSE
> setequal(x,c(1,5))
[1] FALSE
```

```
> 5 %in% x
[1] TRUE
> 5 %in% y
[1] TRUE
> choose(5,2)
[1] 10
```

03

기본적인 R 함수

3. 기본적인 R 함수

▶ 수치적 함수
 <수치함수>

함수	예
•pi	> pi [1] 3.141593 > options(digits=20) > pi [1] 3.141592653589793

함수	예	
•log(x) : 자연로그 함수	예1) > log(2) [1] 0.7	예2) > x <- 3 > y <- 4 > log(x+y) [1] 1.9
•log10(x) : 상용로그 함수	예1) > log10(10) [1] 1	예2) > x <- 3 > y <- 14 > log10(x+y) [1] 1.2
•exp(x) : 지수로그 함수	> exp(10) [1] 22026.5	
•sqrt(x) : 루트함수	> sqrt(8) [1] 2.8	

3. 기본적인 R 함수

<삼각함수>

함수	예
•sin(x) : sine 함수	> sin(10) [1] - 0.54
•cos(x) : cosine 함수	> cos(10) [1] - 0.84
•tan(x) : tangent 함수	> tan(10) [1] 0.65
•asin(x) : arcsine 함수	> asin(1) [1] 1.6
•acos(x) : arccosine 함수	> acos(0) [1] 1.6
•atan(x) : arctangent 함수	> atan(0.6) [1] 0.54

함수	예
•min(x) : 벡터에서 최솟값	> x <- c(1, 2, - 3, 4) > min(x) [1] - 3
•max(x) : 벡터에서 최댓값	> x <- c(1, 2, - 3, 4) > max(x) [1] 4
•min(x1, x2, ...): 전체 벡터원소 중에서 최솟값	> x1 <- c(1, 2, - 3, 4) > x2 <- c(2, 4, - 6, 7) > min(x1, x2) [1] - 6
•range(x) : 벡터의 범위 → c(min(x), max(x))	> x <- c(1, 2, - 3, 4) > range(x) [1] - 3 4 > c(min(x), max(x)) [1] - 3 4

3. 기본적인 R 함수

<벡터의 최소·최대 함수>

함수	예
•pmin(x1, x2): 두 벡터의 상응하는 원소들 중 작은 값	<pre>> x1 <- c(1, 2, - 3, 4) > x2 <- c(2, 4, - 6, 7) > pmin(x1, x2) [1] 1 2 - 6 4</pre>
•pmax(x1, x2): 두 벡터의 상응하는 원소들 중 큰 값	<pre>> x1 <- c(1, 2, - 3, 4) > x2 <- c(2, 4, - 6, 7) > pmax(x1, x2) [1] 2 4 - 3 7</pre>

3. 기본적인 R 함수

▶ 통계함수

- R의 장점 중 한 가지는 다양한 통계함수가 내장되어 있어서 기본적인 통계량 산출이 용이하다는 것임

함수	예
•mean(x1): 평균	> x1 <- c(1, 2, 3, 4, 5, 6) > mean(x1) [1] 3.5
•sd(x1): 표준편차	> x1 <- c(1, 2, 3, 4, 5, 6) > sd(x1) [1] 1.9
•var(x1): 분산	> x1 <- c(1, 3, 6, 9, 12, 3, 2) > var(x1) [1] 16.5
•median(x1): 중앙값(중위수)	> x1 <- c(1, 3, 6, 9, 12, 3, 2) > median(x1) [1] 3

<위치와 산포>

함수	예
•quantile(x, p) : (100 * p)%에 해당하는 값	> x1 <- c(1, 2, 3, 4, 5, 6, 7, 8, 9, 10) > quantile(x1, 0.5) [1] 50% [1] 5.5
•cor(x, y) : 상관계수	> x <- c(1, 2, 3, 4, 5, 6, 7, 8, 9, 10) > y <- c(10, 9, 8, 7, 6, 5, 4, 3, 2, 5) > cor(x, y) [1] - 0.91

<분위수와 상관계수>

04

실습하기

09강

다음시간안내 ▶▶▶

R 프로그래밍의 핵심 구조 (2)